CINEMA RESERVATION SYSTEM



Introduction



This project is about a Cinema Management System. It ,helps the cinema organize movies, schedules, tickets and customer information. The goal is to make the work easier and faster for the cinema. It also gives a better experience for people who buy tickets and watch .movies

BUSINESS RULES

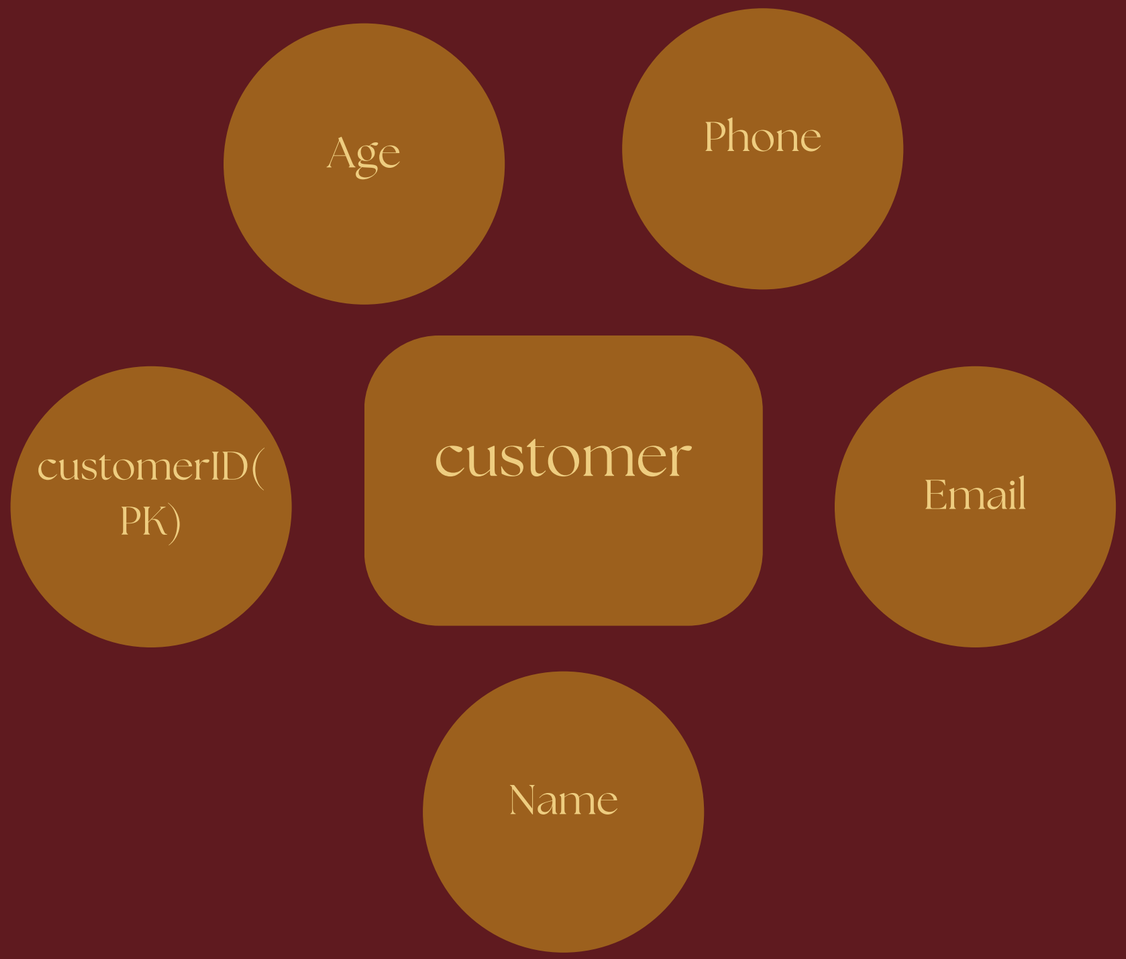
- Customers must not book a movie if their age is below the rating. (age restriction can be done using trigger)

- Valid payment methods include : Cash, Credit Card, Visa, Online (should use CHECK constraint on Type).

- Employees must be older than 17 years (should use CHECK constraint on Age).



CUSTOMER ENTITY





EMPLOYEE ENTITY

EmployeeID(PK)

Age

NAME

EMPLOYEE

Shift

BIRTHDATE

Email

Phone



MOVIE ENTITY





PAYMENT ENTITY

PaymentID(PK)

Payment

Type

PaymentDate

Amount



HALL ENTITY

HallID(PK)

Hall

Capacity

Type



SHOWTIME ENTITY

MovieHallID (PK)

Date

SHOWTIME

StartTime

EndTime



BOOKING ENTITY

BookingID(PK)

Movie

BookingDate



SEAT ENTITY

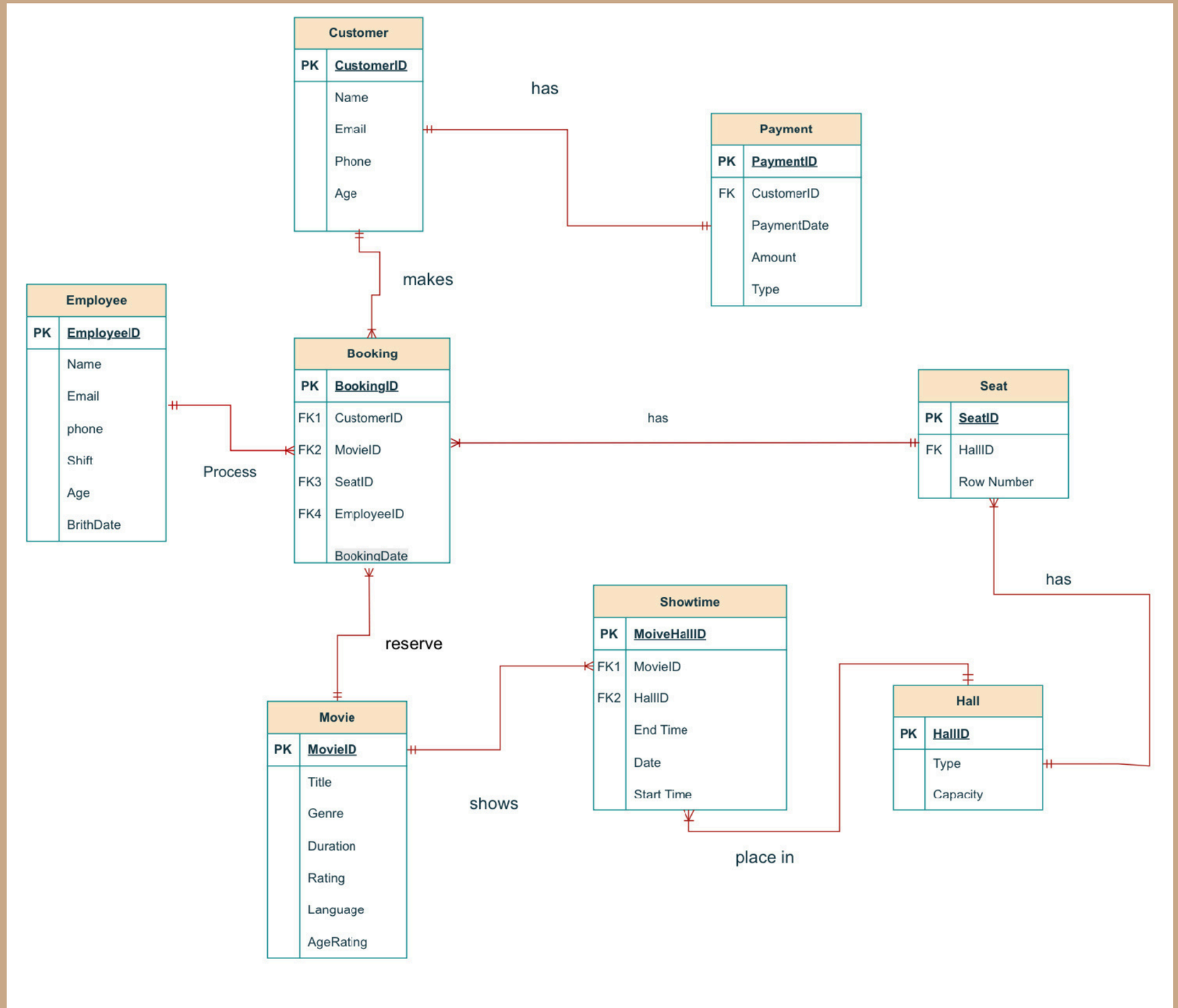
SeatID(PK)

Payment

RowNumber



FIRST ERD


Customer
PK CustomerID
Name
Email
Phone
Age
has
Payment
PK PaymentID
FK CustomerID
PaymentDate
Amount
Type
makes
Employee
PK EmployeeID
Name
Email
phone
Shift
Age
BrithDate
Booking
PK BookingID
FK1 CustomerID
FK2 MovieID
FK3 SeatID
FK4 EmployeeID
BookingDate
Process
has
Seat
PK SeatID
FK HallID
Row Number
has
reserve
Showtime
PK MoiveHallID
FK1 MovieID
FK2 HallID
End Time
Date
Start Time
Movie
PK MovieID
Title
Genre
Duration
Rating
Language
AgeRating
shows
Hall
PK HallID
Type
Capacity
place in

Normalisation Customer table:

Customer:

CustomerID	Name		Email	Phone	Age
	FirstName	LastName			

1NF is not satisfied:

There is composite attribute (Name). To satisfy INF the tables will be as follow:

CustomerID	FirstName	LastName	Email	Phone	Age

Primary Key is defined (CustomerID).

Now the table is in 1NF.

2NF is satisfied:

It's in 1NF and there is no partial dependency, all attributes are fully functionally dependent on the primary key.

3NF is satisfied:

It's in 2NF and there is no transitive dependency, no non-key attribute depends on another non-key attribute.

Normalisation Employee table:

Employee:

EmployeeID	Name		Email	Phone	Shift	Age	BrithDate
	FirstName	LastName					

1NF is not satisfied:

There is composite attribute (Name). To satisfy INF the tables will be as follow:

EmployeeID	FirstName	LastName	Email	Phone	Shift	Age	BrithDate

Primary Key is defined (EmployeeID).

Now the table is in 1NF.

2NF is satisfied:

It's in 1NF and there is no partial dependency, all attributes are fully functionally dependent on the primary key.

3NF is satisfied:

It's in 2NF and there is no transitive dependency, no non-key attribute depends on another non-key attribute.

Normalisation Movie table:

Movie:

MovieID	Title	Genre	Rating	Duration	Language	AgeRating

1NF is satisfied:

1. All Attributes are atomic.
2. Primary Key is defined (MovieID).

2NF is satisfied:

It's in 1NF and there is no partial dependency, all attributes are fully functionally dependent on the primary key.

3NF is satisfied:

It's in 2NF and there is no transitive dependency, no non-key attribute depends on another non-key attribute.

Normalisation Payment table:

Payment:

PaymentID	PaymentDate	Amount	Type

1NF is satisfied:

1. All Attributes are atomic.
2. Primary Key is defined (PaymentID).

2NF is satisfied:

It's in 1NF and there is no partial dependency, all attributes are fully functionally dependent on the primary key.

3NF is satisfied:

It's in 2NF and there is no transitive dependency, no non-key attribute depends on another non-key attribute.

Normalisation Hall table:

Hall:

HallID	Type	Capacity

1NF is satisfied:

1. All Attributes are atomic.
2. Primary Key is defined (HallID).

2NF is satisfied:

It's in 1NF and there is no partial dependency, all attributes are fully functionally dependent on the primary key.

3NF is satisfied:

It's in 2NF and there is no transitive dependency, no non-key attribute depends on another non-key attribute.

Normalisation Showtime table:

Showtime:

MovieHallID	Start Time	End Time	Date

1NF is satisfied:

1. All Attributes are atomic.
2. Primary Key is defined (MovieHallID).

2NF is satisfied:

It's in 1NF and there is no partial dependency, all attributes are fully functionally dependent on the primary key.

3NF is not satisfied:

It's in 2NF and there is no transitive dependency, no non-key attribute depends on another non-key attribute.

Normalisation Booking table:

Booking:

BookingID	BookingDate

1NF is satisfied:

1. All Attributes are atomic.
2. Primary Key is defined (BookingID).

2NF is satisfied:

It's in 1NF and there is no partial dependency, all attributes are fully functionally dependent on the primary key.

3NF is satisfied:

It's in 2NF and there is no transitive dependency, no non-key attribute depends on another non-key attribute.

Normalisation seat table:

Seat:

SeatID	Row Number

1NF is satisfied:

1. All Attributes are atomic.
2. Primary Key is defined (SeatID).

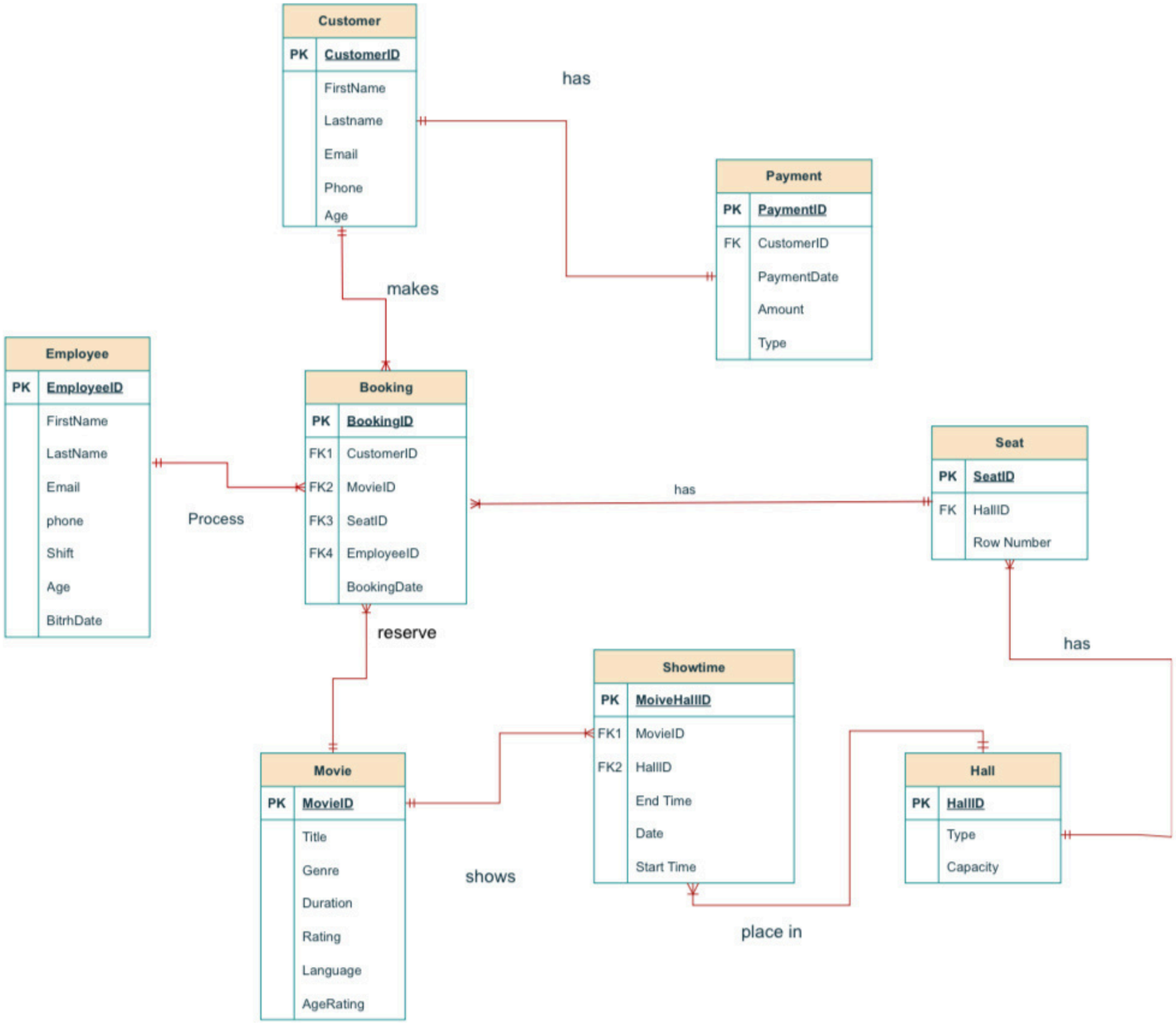
2NF is satisfied:

It's in 1NF and there is no partial dependency, all attributes are fully functionally dependent on the primary key.

3NF is satisfied:

It's in 2NF and there is no transitive dependency, no non-key attribute depends on another non-key attribute.

FINAL ERD


Customer
PK CustomerID
FirstName
Lastname
Email
Phone
Age
has
Payment
PK PaymentID
FK CustomerID
PaymentDate
Amount
Type
makes
Employee
PK EmployeeID
FirstName
LastName
Email
phone
Shift
Age
BitrhDate
Process
Booking
PK BookingID
FK1 CustomerID
FK2 MovieID
FK3 SeatID
FK4 EmployeeID
BookingDate
has
Seat
PK SeatID
FK HallID
Row Number
reserve
has
Showtime
PK MoiveHallID
FK1 MovieID
FK2 HallID
End Time
Date
Start Time
Movie
PK MovieID
Title
Genre
Duration
Rating
Language
AgeRating
shows
Hall
PK HallID
Type
Capacity
place in

IMPLEMENTATION OF CODE

Customer

```sql
-- Customer Table
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    FirstName VARCHAR(20),
    LastName VARCHAR(20),
    Email VARCHAR(50) UNIQUE,
    Phone VARCHAR(12),
    Age INT
);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

CustomerID	FirstName	LastName	Email	Phone	Age
1	Sara	Ali	sara.ali@gmail.com	0551234567	21
2	Fahad	Hassan	fahad.hassan@gmail.com	0562345678	24
3	Layla	Khalid	layla.khalid@gmail.com	0573456789	19
4	Omar	Yousef	omar.yousef@gmail.com	0584567890	25
5	Maha	Saad	maha.saad@gmail.com	0595678901	17
6	Noura	Salem	noura.salem@gmail.com	0506789012	20
NULL	NULL	NULL	NULL	NULL	NULL

```sql
-- Insert 6 sample customers
INSERT INTO Customer
VALUES
(1, 'Sara', 'Ali', 'sara.ali@gmail.com', '0551234567', 21),
(2, 'Fahad', 'Hassan', 'fahad.hassan@gmail.com', '0562345678', 24),
(3, 'Layla', 'Khalid', 'layla.khalid@gmail.com', '0573456789', 19),
(4, 'Omar', 'Yousef', 'omar.yousef@gmail.com', '0584567890', 25),
(5, 'Maha', 'Saad', 'maha.saad@gmail.com', '0595678901', 17),
(6, 'Noura', 'Salem', 'noura.salem@gmail.com', '0506789012', 20);
```

Employee

```sql
-- Employee Table
CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    FirstName VARCHAR(20),
    LastName VARCHAR(20),
    Email VARCHAR(50) UNIQUE,
    Phone VARCHAR(12),
    Shift VARCHAR(10),
    Age INT CHECK (Age >= 18),
    BirthDate DATE
);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

EmployeeID	FirstName	LastName	Email	Phone	Shift	Age	BirthDate
101	Khalid	Ahmed	khalid.ahmed@cinema.com	0501234567	Morning	29	1996-05-12
102	Reem	Faisal	reem.faisal@cinema.com	0512345678	Afternoon	24	2000-02-20
103	Salem	Nader	salem.nader@cinema.com	0523456789	Night	32	1992-07-08
104	Amani	Hassan	amani.hassan@cinema.com	0534567890	Morning	27	1997-01-15
105	Tariq	Zaid	tariq.zaid@cinema.com	0545678901	Afternoon	30	1995-09-10
106	Huda	Kareem	huda.kareem@cinema.com	0556789012	Night	22	2002-11-05
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```sql
-- Insert 6 sample employees
INSERT INTO Employee
VALUES
(101, 'Khalid', 'Ahmed', 'khalid.ahmed@cinema.com', '0501234567', 'Morning', 29, '1996-05-12'),
(102, 'Reem', 'Faisal', 'reem.faisal@cinema.com', '0512345678', 'Afternoon', 24, '2000-02-20'),
(103, 'Salem', 'Nader', 'salem.nader@cinema.com', '0523456789', 'Night', 32, '1992-07-08'),
(104, 'Amani', 'Hassan', 'amani.hassan@cinema.com', '0534567890', 'Morning', 27, '1997-01-15'),
(105, 'Tariq', 'Zaid', 'tariq.zaid@cinema.com', '0545678901', 'Afternoon', 30, '1995-09-10'),
(106, 'Huda', 'Kareem', 'huda.kareem@cinema.com', '0556789012', 'Night', 22, '2002-11-05');
```

Movie

```sql
-- Movie Table
CREATE TABLE Movie (
    MovieID INT PRIMARY KEY,
    Title VARCHAR(50),
    Genre VARCHAR(20),
    Duration TIME,
    Rating VARCHAR(5),
    Language VARCHAR(10),
    AgeRating INT
);
```

```sql
-- Insert 6 sample movies
INSERT INTO Movie
VALUES
(301, 'Edge of Tomorrow', 'Action', '01:53:00', 'M', 'English', 18),
(302, 'Parasite', 'Thriller', '02:12:00', 'M', 'Korean', 15),
(303, 'Soul', 'Animation', '01:40:00', 'G', 'English', 10),
(304, 'Joker', 'Drama', '02:02:00', 'R18+', 'English', 18),
(305, 'Inception', 'Sci-Fi', '02:28:00', 'PG', 'English',12),
(306, 'The Conjuring', 'Horror', '01:52:00', 'R18+', 'English', 18);
```

MovieID	Title	Genre	Duration	Rating	Language	AgeRating
301	Edge of Tomorrow	Action	01:53:00	M	English	18
302	Parasite	Thriller	02:12:00	M	Korean	15
303	Soul	Animation	01:40:00	G	English	10
304	Joker	Drama	02:02:00	R18+	English	18
305	Inception	Sci-Fi	02:28:00	PG	English	12
306	The Conjuring	Horror	01:52:00	R18+	English	18
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Payment

```sql
-- Payment Table with CHECK constraint
CREATE TABLE Payment (
    PaymentID INT PRIMARY KEY,
    CustomerID INT,
    PaymentDate DATE,
    Amount DECIMAL(10,2),
    Type VARCHAR(50) CHECK (Type IN ('Cash', 'Credit Card', 'Visa', 'Online')),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);
```

PaymentID	CustomerID	PaymentDate	Amount	Type
201	1	2025-04-01	45.00	Credit Card
202	2	2025-04-02	60.00	Cash
203	3	2025-04-03	38.50	Online
204	4	2025-04-04	72.00	Visa
205	5	2025-04-05	55.75	Credit Card
206	6	2025-04-06	42.00	Cash
NULL	NULL	NULL	NULL	NULL

```sql
-- Insert 6 sample payments
INSERT INTO Payment
VALUES
(201, 1, '2025-04-01', 45.00, 'Credit Card'),
(202, 2, '2025-04-02', 60.00, 'Cash'),
(203, 3, '2025-04-03', 38.50, 'Online'),
(204, 4, '2025-04-04', 72.00, 'Visa'),
(205, 5, '2025-04-05', 55.75, 'Credit Card'),
(206, 6, '2025-04-06', 42.00, 'Cash');
```

Hall

```sql
-- Hall Table
CREATE TABLE Hall (
    HallID INT PRIMARY KEY,
    Type VARCHAR(30),
    Capacity INT CHECK (Capacity > 0)
);
```

HallID	Type	Capacity
1	Standard	120
2	VIP	60
3	3D	100
4	IMAX	150
5	Deluxe	80
6	Kids	50
NULL	NULL	NULL

```sql
-- Insert 6 sample halls
INSERT INTO Hall
VALUES
(1, 'Standard', 120),
(2, 'VIP', 60),
(3, '3D', 100),
(4, 'IMAX', 150),
(5, 'Deluxe', 80),
(6, 'Kids', 50);
```

Showtim

```sql
-- Showtime Table
CREATE TABLE Showtime (
    MovieHallID INT PRIMARY KEY,
    MovieID INT,
    HallID INT,
    Date DATE,
    StartTime TIME,
    EndTime TIME,
    FOREIGN KEY (MovieID) REFERENCES Movie(MovieID),
    FOREIGN KEY (HallID) REFERENCES Hall(HallID)
);
```

```sql
-- Insert 6 sample showtimes
INSERT INTO Showtime
VALUES
(501, 301, 1, '2025-04-25', '15:00:00', '16:53:00'),
(502, 302, 2, '2025-04-25', '17:00:00', '19:12:00'),
(503, 303, 3, '2025-04-25', '13:30:00', '15:10:00'),
(504, 304, 4, '2025-04-25', '20:00:00', '22:02:00'),
(505, 305, 5, '2025-04-25', '18:00:00', '20:28:00'),
(506, 306, 6, '2025-04-25', '22:00:00', '23:52:00');
```

MovieHallID	MovieID	HallID	Date	StartTime	EndTime
501	301	1	2025-04-25	15:00:00	16:53:00
502	302	2	2025-04-25	17:00:00	19:12:00
503	303	3	2025-04-25	13:30:00	15:10:00
504	304	4	2025-04-25	20:00:00	22:02:00
505	305	5	2025-04-25	18:00:00	20:28:00
506	306	6	2025-04-25	22:00:00	23:52:00
NULL	NULL	NULL	NULL	NULL	NULL

Bookin

```sql
-- Booking Table
CREATE TABLE Booking (
    BookinID INT PRIMARY KEY,
    CustomerID INT,
    MovieID INT,
    SeatID INT,
    EmployeeID INT,
    BookingDate DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (MovieID) REFERENCES Movie(MovieID),
    FOREIGN KEY (SeatID) REFERENCES Seat(SeatID),
    FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID)
);
```

```sql
-- Insert 6 sample bookings
INSERT INTO Booking
VALUES
(601, 1, 301, 401, 101, '2025-04-25'),
(602, 2, 302, 403, 102, '2025-04-25'),
(603, 3, 303, 405, 103, '2025-04-25'),
(604, 4, 304, 404, 104, '2025-04-25'),
(605, 5, 305, 406, 105, '2025-04-25'),
(606, 6, 306, 402, 106, '2025-04-25');
```

BookinID	CustomerID	MovieID	SeatID	EmployeeID	BookingDate
601	1	301	401	101	2025-04-25
602	2	302	403	102	2025-04-25
603	3	303	405	103	2025-04-25
604	4	304	404	104	2025-04-25
605	5	305	406	105	2025-04-25
606	6	306	402	106	2025-04-25
NULL	NULL	NULL	NULL	NULL	NULL

Sea

```sql
-- Seat Table
CREATE TABLE Seat (
    SeatID INT PRIMARY KEY,
    HallID INT,
    RowNumber VARCHAR(10),
    FOREIGN KEY (HallID) REFERENCES Hall(HallID)
);
```

```sql
-- Insert 6 sample seats
INSERT INTO Seat
VALUES
(401, 1, 'A1'),
(402, 1, 'A2'),
(403, 2, 'B1'),
(404, 2, 'B2'),
(405, 3, 'C1'),
(406, 3, 'C2');
```

SeatID	HallID	RowNumber
401	1	A1
402	1	A2
403	2	B1
404	2	B2
405	3	C1
406	3	C2
NULL	NULL	NULL

Trigger

```sql
-- Trigger: Prevent underage customer from booking a restricted movie
DELIMITER //
CREATE TRIGGER trg_prevent_underage_booking
BEFORE INSERT ON Booking
FOR EACH ROW
BEGIN
    DECLARE custAge INT;
    DECLARE movieAgeRating INT;
    SELECT Age INTO custAge FROM Customer WHERE CustomerID = NEW.CustomerID;
    SELECT AgeRating INTO movieAgeRating FROM Movie WHERE MovieID = NEW.MovieID;
    IF custAge < movieAgeRating THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Booking denied: Customer age is below movie age rating.';
    END IF;
END;
//
DELIMITER ;
```

```sql
INSERT INTO customer VALUES (7,'Hanan','Alanazi','hanan2004@gmail.com',0509625573,15);
INSERT INTO Booking VALUES (607,7,301,'407',105,curdate());
```

205 01:21:57 INSERT INTO Booking VALUES (607,7,301,'407',105,curdate()) Error Code: 1644. Booking denied: Customer age is below movie age rating. 0.016 sec

QUERY SAMPLES

```sql
-- Select customers who are allowed to watch 18+ movies
SELECT * FROM Customer
WHERE Age >= 18;
```

CustomerID	FirstName	LastName	Email	Phone	Age
1	Sara	Ali	sara.ali@gmail.com	0551234567	21
2	Fahad	Hassan	fahad.hassan@gmail.com	0562345678	24
3	Layla	Khalid	layla.khalid@gmail.com	0573456789	19
4	Omar	Yousef	omar.yousef@gmail.com	0584567890	25
6	Noura	Salem	noura.salem@gmail.com	0506789012	20
NULL	NULL	NULL	NULL	NULL	NULL

```sql
-- Find movies where the genre is 'Action' AND the age rating is 18
SELECT *
FROM Movie
WHERE Genre = 'Action' AND AgeRating >= 18;
```

MovieID	Title	Genre	Duration	Rating	Language	AgeRating
301	Edge of Tomorrow	Action	01:53:00	M	English	18
NULL	NULL	NULL	NULL	NULL	NULL	NULL

QUERY SAMPLES

```
7     -- Find payments where the type is 'Cash' OR type is 'Visa'
8 •   SELECT *
9     FROM Payment
0     WHERE Type = 'Cash' OR Type = 'Visa';
1
```

PaymentID	CustomerID	PaymentDate	Amount	Type
202	2	2025-04-02	60.00	Cash
204	4	2025-04-04	72.00	Visa
206	6	2025-04-06	42.00	Cash
NULL	NULL	NULL	NULL	NULL

```sql
-- Count how many employees are assigned to each shift
SELECT Shift, COUNT(*) AS NumberOfEmployees
FROM Employee
GROUP BY Shift;
```

Shift	NumberOfEmployees
Morning	2
Afternoon	2
Night	2

QUERY SAMPLES

```sql
-- Get highest payment
SELECT MAX(Amount) AS HighestPayment FROM Payment;
```

	HighestPayment
▶	72.00

```sql
-- Get lowest payment
SELECT MIN(Amount) AS LowestPayment FROM Payment;
```

	LowestPayment
▶	38.50

```sql
-- Calculate average amount paid
SELECT AVG(Amount) AS AveragePayment FROM Payment;
```

	AveragePayment
▶	52.208333

QUERY SAMPLES

```sql
-- Get customer names with their payment info
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    p.PaymentID,
    p.PaymentDate,
    p.Amount,
    p.Type
FROM Customer c
INNER JOIN Payment p ON c.CustomerID = p.CustomerID;
```

CustomerID	FirstName	LastName	PaymentID	PaymentDate	Amount	Type
1	Sara	Ali	201	2025-04-01	45.00	Credit Card
2	Fahad	Hassan	202	2025-04-02	60.00	Cash
3	Layla	Khalid	203	2025-04-03	38.50	Online
4	Omar	Yousef	204	2025-04-04	72.00	Visa
5	Maha	Saad	205	2025-04-05	55.75	Credit Card
6	Noura	Salem	206	2025-04-06	42.00	Cash

QUERY SAMPLES

```sql
-- inner join: Customer ↔ Booking ↔ Seat
SELECT
    c.CustomerID,
    c.FirstName,
    c.LastName,
    p.PaymentID,
    p.Amount,
    s.SeatID,
    s.RowNumber
FROM Customer c
INNER JOIN Payment p ON c.CustomerID = p.CustomerID
INNER JOIN Booking b ON c.CustomerID = b.CustomerID
INNER JOIN Seat s ON b.SeatID = s.SeatID;
```

CustomerID	FirstName	LastName	PaymentID	Amount	SeatID	RowNumber
1	Sara	Ali	201	45.00	401	A1
2	Fahad	Hassan	202	60.00	403	B1
3	Layla	Khalid	203	38.50	405	C1
4	Omar	Yousef	204	72.00	404	B2
5	Maha	Saad	205	55.75	406	C2
6	Noura	Salem	206	42.00	402	A2



THANK YOU FOR LISTENING

ANY QUQUESTION?